# Recommender System

CSDS 335/435 Data Mining

Maura Schorr
Case Western Reserve University
Computer and Data Sciences
Cleveland, OH, United States
mes369@case.edu

Jacob Anderson
Case Western Reserve University
Computer and Data Sciences
Cleveland, OH, United States
jma195@case.edu

## 1 Introduction

Recommendation systems are becoming increasingly relevant in the digital age to provide personalized content on e-commerce dashboards, social media feed, and video streaming platforms. Our analysis compares the performance of five algorithms for recommender systems using movie ratings from the MovieLens small dataset: random, k nearest neighbors (kNN), singular value decomposition (SVD), matrix factorization with regularization, and a deep neural network. The random algorithm serves as a baseline upon which the four collaborative filtering algorithms can be comparatively evaluated. Each algorithm is evaluated using the same 5-fold cross-validation data partitions, and performance for each is reported as root mean squared error (RMSE), mean absolute error (MAE), and runtime.

## 2 Algorithms

### 2.1 Random

*2.1.1 Theoretical Motivation.* The random algorithm is a baseline upon which to compare the other algorithms. In our analysis, we use the Surprise library's NormalPredictor implementation, which fits a Gaussian distribution to the training ratings and samples predicted values for each test pair.

Formally, given a training set $\mathcal{R} = \{r_{ui}\}$ of ratings for users $u$ and movie $i$, parameters are fit to the empirical distribution using maximum likelihood: $\mu = \frac{1}{|\mathcal{R}|}\sum_{(u,i)\in\mathcal{R}} r_{ui}$, and $\sigma^2 = \frac{1}{|\mathcal{R}|}\sum_{(u,i)\in\mathcal{R}}(r_{ui}-\mu)^2$. Then, for each test pair $(u,i) \in \mathcal{T}$, a predicted rating is sampled from the distribution as $\hat{r}_{ui} \sim \mathcal{N}(\mu, \sigma^2)$, and the sample is then clipped to the valid range for ratings, which in this dataset is between 0.5 and 5.0.

*2.1.2 Hyperparameters.* The only hyperparameter involved in the Random algorithm is the choice of distribution. This selection could be based on the observed rating distribution of the data, but the Surprise implementation currently supports only the Gaussian. Nonetheless, for an attribute that is performance-based in nature, a bell-curve distribution assumption is the most appropriate.

### 2.2 k-Nearest Neighbors (kNN)

*2.2.1 Theoretical Motivation.* Our analysis used the user generated 'tags', or words describing the movies to recommend similar movies. kNN used the movie tags to compare the similarities between the movies, so it was not a user-based implementation. We implemented Surprise library's KNNBasic using cosine distance.

How kNN works is it first converts each movie into a vector using the tags that describe it. Next it compares two movies to find how similar they are, we used cosine distance here because it is good for sparse datasets. Then the algorithm finds the k most similar movies and takes the ratings of these neighbor movies and computes a weighted average to produce the predicted rating. The algorithm repeats this process for every user-movie pair in the dataset.

*2.2.2 Hyperparameters.* The hyperparameter in kNN is k, the number of nearest neighbors that will be taken into account when making predictions. After testing multiple values of k and comparing the RMSE and MAE for each one, we found that k=80 was the best k value to use. This tells us that the model performs better when it can smooth predictions over many neighbors.

### 2.3 Singular Value Decomposition (SVD)

*2.3.1 Theoretical Motivation.* The prediction for each user-item rating is given as a linear combination of latent factor vectors learned for the user and item and learned bias terms for each item and user, $\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$, where $\mu$ is the empirical mean rating. The bias terms themselves represented the tendency of some user to rate above or below the empirical average rating and the tendency of some item to be rated above or below the empirical average rating, respectively.

These parameters are estimated by minimizing the regularized squared error over the ratings in the training set, $\min_{b_u,b_i,p_u,q_i} \sum_{(u,i)\in\mathcal{R}} (r_{ui} - \hat{r}_{ui})^2 + \lambda\left(b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2\right)$, where $\lambda$ represents the regularization strength controlling the penalty on the parameter magnitude, as a means to prevent overfitting on the data. To find a local optimum for this objective, a prediction error is computed for each rating as $e_{ui} = r_{ui} - \hat{r}_{ui}$ and the parameters are updated accordingly:

$$b_u \leftarrow b_u + \gamma\left(e_{ui} - \lambda b_u\right)$$

$$b_i \leftarrow b_i + \gamma\left(e_{ui} - \lambda b_i\right)$$

$$p_u \leftarrow p_u + \gamma\left(e_{ui} \cdot q_i - \lambda p_u\right)$$

$$q_i \leftarrow q_i + \gamma\left(e_{ui} \cdot p_u - \lambda q_i\right)$$

where $\gamma$ is the learning rate hyperparameter. Each update is applied in series for each of the examples in the training set to constitute one epoch. Following training, a prediction for each

test pair $(u, i) \in \mathcal{T}$ again is given by $\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$, which is performed for each fold.

*2.3.2 Hyperparameters.* This algorithm is dicated by four hyperparameters: the choice of latent factor dimension $k$, the number of epochs, the learning rate $\gamma$, and the regularization strength $\lambda$. $k$ controls the dimensionality of the latent space and balances a more complex item-user interaction with risk of overfitting, $\gamma$ controls the step size of each SGD update and will determine how efficiently the SGD can converge toward an optimum, and the setting of $\lambda$ balances the cost of overfitting with potentially underfitting if the regularization is too strong.

These hyperparameters were set for the cross validation by a search over the following space, using average RMSE across folds as the guiding heuristic:

$$k \in \{50, 100, 150, 200\}$$

$$\text{epochs} \in \{20, 50, 100\}$$

$$\gamma \in \{0.002, 0.005, 0.01\}$$

$$\lambda \in \{0.02, 0.05, 0.1\}$$

The optimal settings given by the search were $k = 200$, epochs $= 100$, $\gamma = 0.01$, and $\lambda = 0.1$. The search suggest that the model benefits from the largest latent representation to capture user and item characteristics coupled with a high number of epochs with which to produce a model of the user-item interactions. The optimization space is amenable to large parameter updates, and the strong regularization is likely necessary to prevent the model from overfitting.

## 2.4 Matrix Factorization with Regularization

*2.4.1 Theoretical Motivation.* Matrix factorization with regularization is similar to SVD but does not include biases, or baselines, with which to model the predicted ratings, instead relying on only the latent factor vectors, $\hat{r}_{ui} = p_u^\top q_i$.

Regularization via the sum of squared latent factor vector norms is used to prevent overfitting and improve generalization. We again used Surprise's SVD implementation, changing 'biased = False' to exclude an baselines.

*2.4.2 Hyperparameters.* The hyperparameters we tested for our matrix factorization were the number of factors and the regularization strength. The combination that yielded the best average RMSE were 10 factors and a 0.02 regularization strength. Without the mean and biases terms, we see that the model benefits from a low-dimensional latent representation, which limits its capacity to memorize patterns in the training set, and through this implicit regularization, the model is most amenable to a low regularization strength.

## 2.5 Deep Neural Network

*2.5.1 Theoretical Motivation.* In the deep neural network algorithmm, users and movies are represented by learned vectors in a shared latent space, which are passed through a neural network structure to produce a rating prediction. First, the MovieLens user and movie IDs are remapped to a 0-based contiguous index range, and from these IDs, embeddings are retrived from a lookup tables at the beginning of the network: $e_u = E_U[u] \in \mathbb{R}^d, \quad e_i = E_I[i] \in \mathbb{R}^d$. These two embeddings are concatenated and form the input to the network, $x_{ui} = [e_u \,\|\, e_i] \in \mathbb{R}^{2d}$.

A dropout layer with rate $p$ is applied, $h^{(0)} = \text{Dropout}(x_{ui},\ p)$, and the result is passed to a sequence of fully-connected layers, with each followed by a ReLU activation and another dropout layer with the same rate, $h^{(\ell)} = \text{Dropout}(\text{ReLU}(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}),\ p), \quad \ell = 1, \ldots, L$. The representation is then passed through a linear layer and a sigmoid activation that is scaled to the valid range of ratings, as $\hat{r}_{ui} = 0.5 + 4.5 \cdot \sigma(W^{(L+1)} h^{(L)} + b^{(L+1)})$.

The network minimizes MSE over each training fold and parameter updates are performed using Adam optimization. The updates are faciliated by mini-batch SGD, where loss over each batch of ratings $\mathcal{B}$ is given as $L = \frac{1}{|B|} \sum_{(u,i) \in B} (r_{ui} - \hat{r}_{ui})^2$. The gradient of $L$ with respect to all parameters is computed and the Adam update is applied. This process repeats over all batches in the training set constituting one epoch. Once parameter estimations were produced, a prediction for each test pair $(u, i) \in \mathcal{T}$ is given by a forward pass through the network, repeated for each fold.

*2.5.2 Hyperparameters.* The hyperparameters can be split into two classes: those related to network structure and those related to the training. Regarding the network structure, the embedding dimension controls how complex the interactions of latent representations may be, and the hidden layer configuration controls the depth and width of the MLP applied to the embeddings. With an optimal architecture selected, a tuned $\alpha$ controls the ability of the network to achieve convergence, dropout rate $p$ will help mitigate complex weight dependencies in the network, and $|B|$ will balance the noise of small gradient updates with the ability to escape local minima and achieve better generalization.

Hyperparameter selection is performed in two stages. In each, selections in the search space produce an average RMSE across folds in a 5-fold cross-validation, and the optimally minimal selection is used in the final CV. The first stage searches for embedding dimesion $d$ and a hidden layer structure over the following search space, with training settings fixed at $\alpha = 0.001$, $p = 0.2$, and $|B| = 256$:

$$d \in \{32, 64, 128\}$$

$$\text{hidden layers} \in \{[128, 64],\ [256, 128],\ [256, 128, 64]\}$$

The second stage searches for $\alpha$, $p$, and $|B|$ over the following space for the optimal architecture found in the first search:

$$\alpha \in \{0.001, 0.005, 0.01\}$$

$$p \in \{0.1, 0.2, 0.4\}$$

$$|B| \in \{128, 256, 512\}$$

The architecture search selected $d = 64$ with hidden layers $[128, 64]$, achieving a mean RMSE of 0.8765. The model appears to benefit from some reduction in capcity to address overfitting behavior.

The optimizer search identified $\alpha = 0.001$, $p = 0.4$, and $|B| = 128$ as the optimal settings for the given architecture, reporting a mean RMSE of 0.8672. The search selected the smallest learning rate and batch size alongside with the highest dropout rate, so the architecture is more performant under conditions where the parameter updates are very stable, small, and heavily constrained so as not the produce dependencies between other weights in the network.

# 3 Results

## 3.1 RMSE and MAE by Algorithm

RMSE and MAE were computed between the sample set and the predicted ratings for each algorithm:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i)\in\mathcal{T}} (r_{ui} - \hat{r}_{ui})^2}$$

$$\text{MAE} = \frac{1}{|\mathcal{T}|} \sum_{(u,i)\in\mathcal{T}} |r_{ui} - \hat{r}_{ui}|$$

Table 1 gives the performance metrics for each fold across all five algorithms.

| Fold | RMSE | MAE | Time |
|---|---|---|---|
| *Random* | | | |
| 1 | 1.4278 | 1.1399 | 0:00:00 |
| 2 | 1.4282 | 1.1346 | 0:00:00 |
| 3 | 1.4184 | 1.1322 | 0:00:00 |
| 4 | 1.4342 | 1.1472 | 0:00:00 |
| 5 | 1.4358 | 1.1485 | 0:00:00 |
| Mean | 1.4289 | 1.1405 | 0:00:00 |
| *kNN* | | | |
| 1 | 0.9716 | 0.7541 | 0:00:09 |
| 2 | 0.9585 | 0.7454 | 0:00:08 |
| 3 | 0.9544 | 0.7435 | 0:00:08 |
| 4 | 0.9703 | 0.7554 | 0:00:08 |
| 5 | 0.9675 | 0.7505 | 0:00:08 |
| Mean | 0.9645 | 0.7498 | 0:00:08 |
| *SVD* | | | |
| 1 | 0.8559 | 0.6536 | 0:00:03 |
| 2 | 0.8439 | 0.6458 | 0:00:03 |
| 3 | 0.8372 | 0.6419 | 0:00:03 |
| 4 | 0.8450 | 0.6477 | 0:00:03 |
| 5 | 0.8538 | 0.6538 | 0:00:03 |
| Mean | 0.8472 | 0.6486 | 0:00:03 |
| *Matrix Factorization* | | | |
| 1 | 0.9465 | 0.7234 | 0:00:00 |
| 2 | 0.9378 | 0.7175 | 0:00:00 |
| 3 | 0.9434 | 0.7199 | 0:00:00 |
| 4 | 0.9285 | 0.7086 | 0:00:00 |
| 5 | 0.9502 | 0.7218 | 0:00:00 |
| Mean | 0.9413 | 0.7183 | 0:00:00 |
| *Neural Network* | | | |
| 1 | 0.8733 | 0.6724 | 0:00:38 |
| 2 | 0.8661 | 0.6721 | 0:00:36 |
| 3 | 0.8612 | 0.6718 | 0:00:36 |
| 4 | 0.8618 | 0.6623 | 0:00:42 |
| 5 | 0.8736 | 0.6792 | 0:00:43 |
| Mean | 0.8672 | 0.6716 | 0:00:39 |

**Table 1: RMSE, MAE, and runtime per fold for each algorithm.**

## 3.2 Pairwise RMSE

RMSE was computed between the predicted ratings of all possible pairs of the five algorithms.

| | Random | kNN | SVD | MF | NN |
|---|---|---|---|---|---|
| **Random** | 0.0000 | 1.0961 | 1.1311 | 1.2291 | 1.1352 |
| **kNN** | 1.0961 | 0.0000 | 0.4517 | 0.6537 | 0.4403 |
| **SVD** | 1.1311 | 0.4517 | 0.0000 | 0.4348 | 0.2347 |
| **MF** | 1.2291 | 0.6537 | 0.4348 | 0.0000 | 0.4367 |
| **NN** | 1.1352 | 0.4403 | 0.2347 | 0.4367 | 0.0000 |

**Table 2: Pairwise RMSE between predicted ratings of each algorithm pair.**

# 4 Discussion

## 4.1 Cross-Validation Performance

SVD achieved the best result on the dataset with an average RMSE of 0.4872, followed closely by the neural network at 0.8672. The matrix factorization and kNN algorithms produced intermediate results at 0.9413 and 0.9645 respectively, and the random baseline established a baseline RMSE of 1.4289. The neural network's additional capacity did not provide an advantage of the classical methods, likely due the size of the dataset and in turn the overfitting constraints that the network had to contend with, as well as the fact that this dataset is known to be well-suited to linear factor models. Nonetheless, the matrix factorization methord performed significantly worse than SVD, indicating that the bias terms and global mean setting of the predictions are valuable for modeling the user-item interractions within the data. The kNN method performed worse than the other collaborative filtering methods, which was expected given how limiting the tag-based similarity is relative to the complexity of the rating signal.

## 4.2 Pairwise RMSE

As expected, the largest pairwise RMSE values involve the random algorithm, which given the performance measures indicates that the prediction behavior is quite different from the trained algorithms. Among the other four, SVD and neural network produce the most similar predictions at 0.2347, which suggest that the neural network is learning a similar latent factor representation of the rating data; however, given its performance, it is likely not as well-optimized due to its respective constraints. The SVD and matrix factorization algorithms are noticeably different at 0.4348, though the absence of the bias terms produces a noticeably degraded and different prediction signal. The two most different algorithms aside from the baseline were the kNN and matrix factorization at 0.6537, which utilize completely different signals and assumptions in producing a rating model.

# 5 Conclusion

SVD produced the best predictive performance on the MovieLens dataset and confirms that linear latent factor models are most amenable to predicting user ratings, as we saw with the Netflix prize. The pairwise RMSE analysis showed that the neural network however does well to produce a similar prediction signal as SVD, even more so than the matrix factorization. Given this quality, future work on this project should include an ablation of each term in the SVD model to

determine which components are most important relative to the prediction signal, and additional features provided to the neural network such as movie genres and user demographics would serve as a more meaningful comparison to the SVD model.

## 6 Author Contributions

Maura Schorr contributed 50% of the work.
Jacob Anderson contributed 50% of the work.
All team members agree with the specified effort.